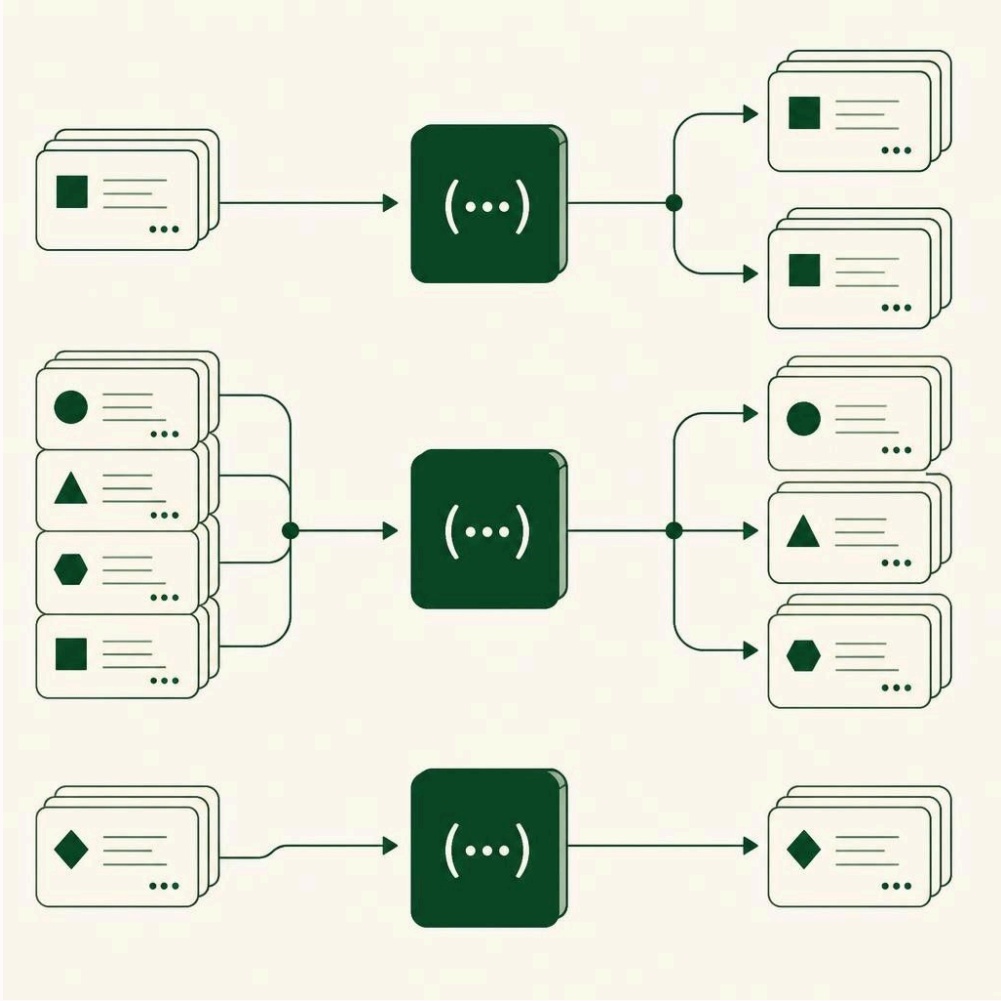
Struct e Funzioni in C

Struct per valore, puntatori a struct, array di struct e funzioni

LINGUAGGIO C | STRUTTURE E UNIONI | STRUCT E FUNZIONI



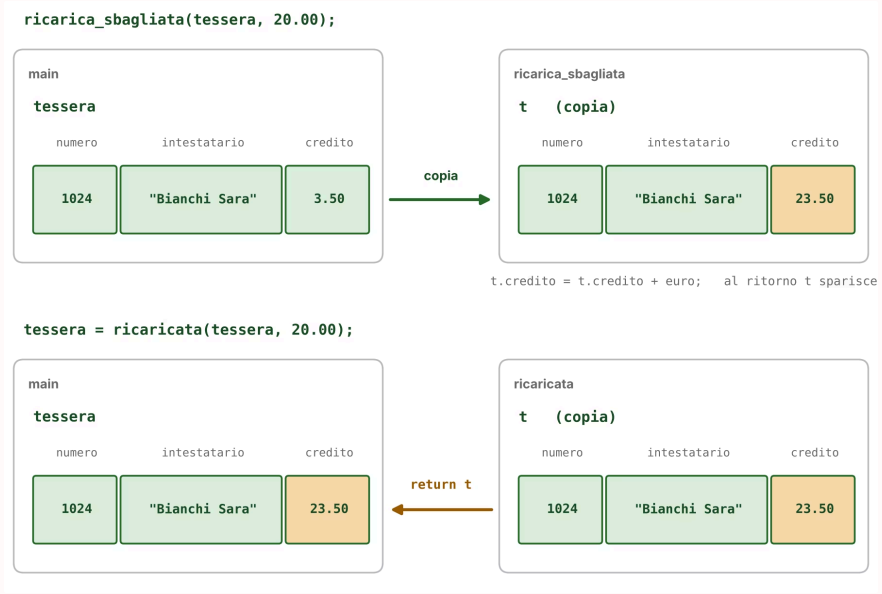
Il Parametro è una Copia

Una struct si passa a una funzione come un int: il parametro riceve una copia di tutti i campi, compresi gli array contenuti nella struct. Le modifiche fatte sulla copia non arrivano al chiamante.

```
void ricarica_sbagliata(Tessera t, double euro);
Tessera ricaricata(Tessera t, double euro);

ricarica_sbagliata(tessera, 20.00);     /* tessera non cambia */
tessera = ricaricata(tessera, 20.00);   /* corretto */
```

> Per cambiare la struct del chiamante ci sono due strade: restituire la struct modificata oppure passare il suo indirizzo.



Funzioni che Restituiscono una Struct

```
Frazione crea(int num, int den);
Frazione somma(Frazione a, Frazione b);
Frazione prodotto(Frazione a, Frazione b);
Frazione semplifica(Frazione f);

Frazione latte = crea(1, 2);
Frazione panna = crea(1, 3);
Frazione dose = semplifica(somma(latte, panna));   /* 5/6 */
```

Con return una funzione restituisce un solo valore: se il valore è una struct, restituisce tutti i suoi campi in un colpo solo. Gli operandi non cambiano, ogni operazione produce un valore nuovo.

> È il modo naturale per i tipi che si comportano come numeri: Frazione, Orario, Complesso. Le chiamate si annidano come le funzioni matematiche.

Il Costo della Copia

```
typedef struct {
    char classe[4];
    int voti[MAX_VOTI];      /* MAX_VOTI = 200 */
    int n_voti;
} Registro;              /* 808 byte */

double media(Registro r);   /* 808 byte copiati a ogni chiamata */
```

> Tre chiamate per valore copiano 2424 byte. Un indirizzo occupa sempre 8 byte, qualunque sia la struct.

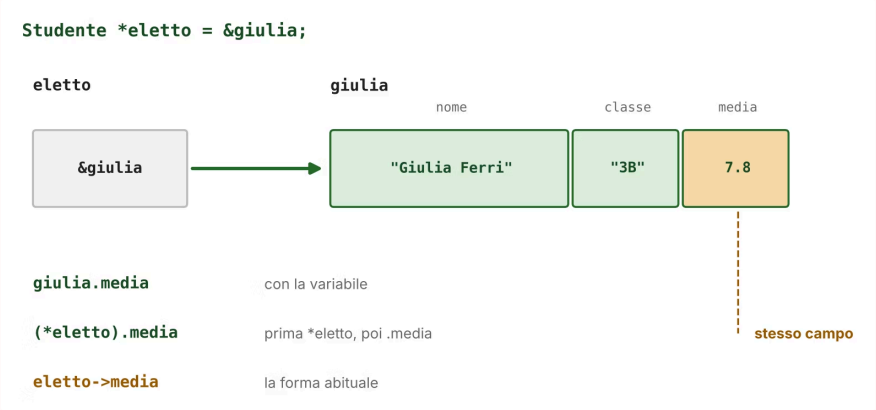
Struct	Dimensione	Per valore?
Frazione	8 byte	sì
Orario	8 byte	sì
Tessera	32 byte	sì
Registro	808 byte	no: si passa l'indirizzo

Puntatore a Struct e Operatore Freccia

Un puntatore può contenere l'indirizzo di una struct. Per raggiungere un campo attraverso il puntatore si usa l'operatore freccia.

```
Studente giulia = {"Giulia Ferri", "3B", 7.8};
Studente *eletto = &giulia;

(*eletto).media     /* prima *eletto, poi .media */
eletto->media       /* la stessa cosa, più leggibile */
```



> Le parentesi in (*eletto).media servono: il punto ha la precedenza, e *eletto.media significherebbe *(eletto.media). Con una variabile si usa il punto, con un puntatore la freccia.

Modificare la Struct del Chiamante

```
void ricarica(Tessera *t, double euro)
{
    t->credito = t->credito + euro;
}

int paga(Tessera *t, double prezzo)
{
    int riuscito = t->credito >= prezzo;

    if (riuscito) {
        t->credito = t->credito - prezzo;
    }
    return riuscito;
}

ricarica(&tessera, 20.00);
```

Per modificare un int del chiamante una funzione riceve un int *; per modificare una struct riceve l'indirizzo della struct.

- non si copia la struct, si passano 8 byte
- le modifiche arrivano al chiamante senza return
- il valore restituito resta libero per l'esito

> paga restituisce 1 se il pagamento è riuscito e 0 se il credito non basta: in quel caso la tessera non cambia.

Leggere senza Copiare: const

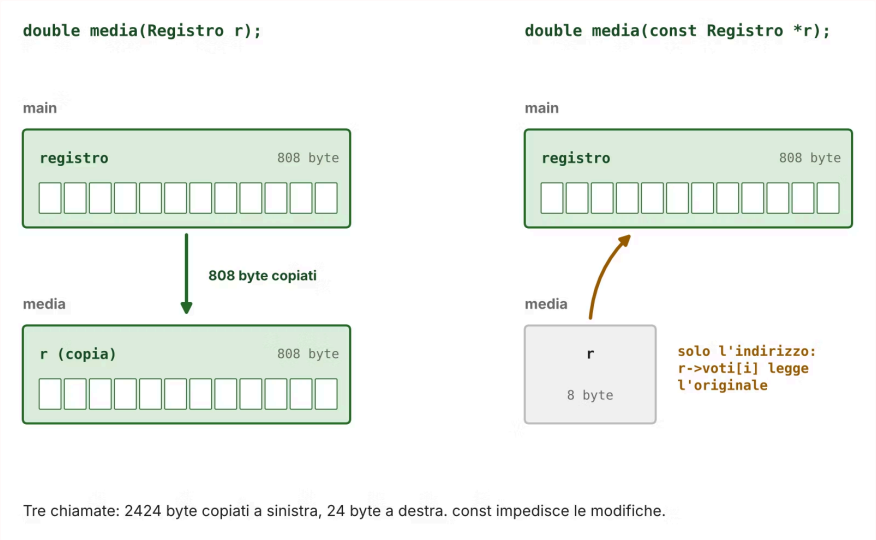
Una funzione che deve solo leggere una struct grande riceve il suo indirizzo con const: niente copia, e il compilatore impedisce le modifiche.

```
double media(const Registro *r);
void aggiungi_voto(Registro *r, int voto);

media(&registro);

r->voti[i]        /* lettura permessa */
r->n_voti = 0;    /* errore: r punta a dati const */
```

> Il prototipo dice al lettore che cosa fa la funzione: const Registro * legge soltanto, Registro * modifica.



Tre chiamate: 2424 byte copiati a sinistra, 24 byte a destra. const impedisce le modifiche.

Tre Prototipi e la Ricerca con Puntatore

Prototipo	Copia	Modifica	Quando
void f(Tipo t)	sì	solo la copia	struct piccole
void f(const Tipo *t)	no	no	sola lettura
void f(Tipo *t)	no	sì	aggiornare la struct

```
Computer *cerca(Computer elenco[], int n,
                const char *inventario);

Computer *pc = cerca(laboratorio, PC, "PC-014");
if (pc != NULL) {
    pc->ram_gb = 16;  /* modifica l'elemento dell'array */
}
```

> Una ricerca può restituire il puntatore all'elemento trovato, o NULL: il chiamante lo modifica direttamente, senza indici.

Array di Struct come Parametro

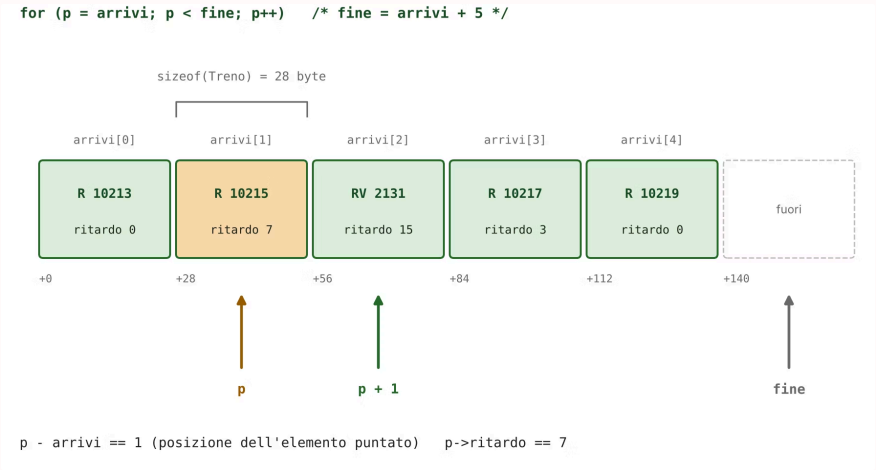
La funzione riceve l'indirizzo del primo elemento; il numero di elementi va passato a parte. Con un puntatore si può scorrere l'array: p + 1 è sizeof(Treno) byte più avanti.

```
double media_classe(const Studente classe[], int n);
void arrotonda(Studente classe[], int n);

const Treno *p;
const Treno *fine = arrivi + TRENI;

for (p = arrivi; p < fine; p++) {
    ... p->ritardo ...
}
```

> const Studente classe[] equivale a const Studente *classe. La posizione dell'elemento puntato è p - arrivi.



Il Criterio come Parametro

```
typedef int (*Criterio)(const Studente *s);

int insufficiente(const Studente *s);
int eccellente(const Studente *s);

int conta(const Studente classe[], int n, Criterio c)
{
    int i;
    int conteggio = 0;

    for (i = 0; i < n; i++) {
        if (c(&classe[i])) {
            conteggio++;
        }
    }
    return conteggio;
}

conta(classe, STUDENTI, insufficiente);
```

Molte domande su un elenco hanno la stessa forma: quanti studenti soddisfano una condizione? La condizione cambia, il ciclo no. La condizione diventa una funzione che riceve uno studente.

> Si passa il nome della funzione, senza parentesi. Un array di Criterio con le descrizioni permette di provarli tutti con un ciclo, come i filtri del catalogo.

Errori Comuni da Evitare

1

Aspettarsi che una funzione che riceve Tessera t modifichi la tessera del chiamante.

2

Chiamare semplifica(f) senza usare il valore restituito.

3

Passare per valore una struct grande dentro un ciclo.

4

Usare il punto con un puntatore (p.media) o la freccia con una variabile (s->media).

5

Scrivere *p.media invece di (*p).media o p->media.

6

Passare tessera invece di &tessera a una funzione che vuole Tessera *.

7

Usare il puntatore restituito da una ricerca senza controllare NULL.

8

Calcolare il numero di elementi con sizeof dentro la funzione che riceve l'array.

Riepilogo Concetti

Situazione	Tecnica	Note
Struct piccola, sola lettura	void f(Tipo t)	si lavora su una copia
Nuovo valore dello stesso tipo	Tipo f(Tipo a, Tipo b)	gli operandi non cambiano
Struct grande, sola lettura	void f(const Tipo *t)	8 byte, nessuna modifica
Aggiornare una struct	void f(Tipo *t)	t->campo = ...
Accedere a un campo	s.campo, p->campo	p->campo è (*p).campo
Ricerca	Tipo *cerca(...)	NULL se non trovato
Array di struct	f(const Tipo v[], int n)	n passato a parte
Scorrere	for (p = v; p < v + n; p++)	p - v è la posizione
Condizione variabile	typedef int (*Criterio)(const Tipo *)	si passa il nome della funzione

Esempi Pratici: Struct per Valore

Concetti base — tre file su copia e struct restituita.

1 – 010-parametro-per-valore.c

Problema	Tecnica
la ricarica della tessera della mensa.	ricarica_sbagliata sulla copia, poi Tessera ricaricata(...).

2 – 020-struct-restituita.c

Problema	Tecnica
le dosi di una ricetta in frazioni di tazza.	Frazione restituita e chiamate annidate.

3 – 030-costo-della-copia.c

Problema	Tecnica
il registro della 3B con fino a 200 voti.	sizeof(Registro) e byte copiati per valore.

Esempi Pratici: Problemi su Struct per Valore

040 – Calcolatrice di frazioni

Le quattro operazioni con il risultato normalizzato.

050 – Orari dei treni

Orario restituito da aggiungi_minuti: i pendolari arrivano in tempo?

060 – Banchi del laboratorio

Rettangoli sulla pianta: sovrapposizione e intersezione.

070 – Cambio valuta

Importo convertito in euro con la tabella dei cambi.

080 – Impedenze di un circuito

Numeri complessi per un circuito RLC in serie e in parallelo.

SEZIONE 2

Esempi Pratici: Puntatori a Struct

Concetti base — tre file su freccia, modifica e const.

1 – 010-operatore-freccia.c

Problema	Tecnica
il rappresentante di classe scelto tra due studenti.	(*p).media e p->media sulla stessa struct.

2 – 020-modificare-tramite-puntatore.c

Problema	Tecnica
la tessera della mensa.	ricarica e paga ricevono Tessera *.

3 – 030-const-per-leggere.c

Problema	Tecnica
lo stesso registro della 3B.	const Registro *, 8 byte invece di 808.

SEZIONE 2

Esempi Pratici: Problemi su Puntatori a Struct

040 – Borsellino elettronico

Acquisti ai distributori e ricariche con codici di esito.

050 – Termostato delle aule

La stessa funzione aggiorna con isteresi due aule diverse.

060 – Scambio di studenti

Scambio tra 1A e 1B tramite puntatori, confronto con le copie.

070 – Lettura di una scheda

leggi_libro riempie la struct del chiamante e controlla l'ISBN.

080 – Ricerca con puntatore

cerca restituisce il PC del laboratorio da modificare.

SEZIONE 3

Esempi Pratici: Array di Struct e Funzioni

Concetti base — tre file su array, puntatori e criteri.

1 – 010-array-come-parametro.c

Problema	Tecnica
le medie di fine anno della classe.	const Studente classe[] e arrotondamento al mezzo voto.

2 – 020-puntatore-che-scorre.c

Problema	Tecnica
il tabellone degli arrivi in stazione.	const Treno *p che scorre e piu_in_ritardo.

3 – 030-criterio-come-parametro.c

Problema	Tecnica
insufficienze, eccellenze e assenze della classe.	typedef Criterio, conta ed elenca.

SEZIONE 3

Esempi Pratici: Problemi su Array di Struct e Funzioni

040 – Tabellone della stazione

Ritardi, soppressioni e prossimo treno per una destinazione.

050 – Campionato di calcetto

Risultati, tabella e capolista senza ordinare l'array.

060 – Noleggio bici

Funzioni che lavorano su una bici trovata con un puntatore.

070 – Elezione dei rappresentanti

Spoglio delle schede, i due più votati e il ballottaggio.

080 – Filtri del catalogo

Ricerche nel catalogo con una tabella di Filtro.